MOTOTRACK

Production Readiness & Security Review

Static analysis of the supplied Laravel source archive, implementation plans, requirements, UI code, authentication flow, and reservation logic.

Overall production readiness	4.5 / 10
Primary blockers	OTP enforcement, public debug exposure, reservation state model, 24-hour automation
Review type	Static code + requirements reconciliation
Report date	October 7, 2026

Key conclusion: MotoTrack has a promising architectural foundation and a substantially improved interface, but the supplied version should not be deployed to production until the authentication gate, reservation lifecycle, and critical information-disclosure issues are corrected.

Table of Contents

1. Executive Summary 4

2. Scope and Review Limitations 4

3. Production-Blocking Findings 5

3.1 Public debug-log endpoint . . . 5
3.2 Public stack traces . . . 5
3.3 OTP middleware may not be enforcing the second factor . . . 5
3.4 Pure Reservation and 24-hour policy are not implemented end to end . . . 5

4. Security and Authentication Review 5

4.1 OTP verification and resend are not rate-limited . . . 5
4.2 OTP generation should use a cryptographically secure generator . . . 6
4.3 OTPs are stored in plaintext . . . 6
4.4 SMTP delivery failures can strand users . . . 6
4.5 Queen SMTP configuration cannot be verified from the archive . . . 6
4.6 Google login bypasses the Email OTP flow . . . 6
4.7 Password login throttling is substantially correct . . . 6
4.8 Unsafe database role default . . . 6
4.9 Role model does not fully match documentation . . . 6
4.10 CSP is present but weakened . . . 7

5. Reservation, Pre-order, and 24-Hour Business Logic 7

5.1 Pre-order flow still contains old payment logic . . . 7
5.2 Client-controlled price in pre-order request . . . 7
5.3 Stale downpayment JavaScript can throw a runtime error . . . 7
5.4 The 24-hour policy currently exists as UI text, not server behavior . . . 7
5.5 Clarify when the 24-hour window starts . . . 8
5.6 Checkout records reservations as completed sales . . . 8
5.7 Customer cancellation conflicts with the sale default . . . 8
5.8 Pre-order completion is not idempotent . . . 8
5.9 Stock reservation semantics should be clearer . . . 8
5.10 User-facing requirements are contradictory . . . 8

6. Architecture and Laravel Best Practices 9

6.1 Positive architectural improvements . . . 9
6.2 Modular routes are a good direction . . . 9
6.3 Multi-shop scaffolding is not yet scoped . . . 9

7. UI/UX and Frontend Review 9

7.1 Navigation and responsive layout are substantially improved . . . 9
7.2 "Single-screen" overview is not guaranteed . . . 9
7.3 Alpine Collapse plugin appears missing . . . 10
7.4 Tailwind major versions appear mixed . . . 10

8. Authorization and Endpoint Review **10**

8.1 Owner and customer access controls are generally well-scoped . . . 10
8.2 Formal REST API is not present in the supplied archive . . . 10
8.3 Search throttling is present but generous . . . 10

9. Performance and Scalability Observations **10**

9.1 Notification queries in Blade . . . 10
9.2 Dashboard history can grow without bounds . . . 10
9.3 Existing positive performance controls . . . 10

10. Recommended Production Remediation Sequence **11**

11. Minimum Test Coverage Before Production **11**

12. Final Verdict **11**

Appendix A. Key Files Referenced **12**

1. Executive Summary

The supplied MotoTrack source archive shows meaningful progress from the earlier architectural audit. The project now includes service abstractions, policies, FormRequests, transactional inventory updates, row locking, modular routes, ULID transaction references, and a more mature custom UI. These changes demonstrate a stronger Laravel architecture than a basic CRUD implementation.

However, the current implementation does not yet enforce several security and business requirements described in the implementation plans. The most serious concerns are a public debug-log endpoint, incomplete enforcement of Email OTP, missing OTP throttling, Google login bypassing OTP, and a reservation flow that still behaves like the older downpayment/sale model instead of a pure reservation system with automatic 24-hour expiration.

CRITICAL: Do not deploy the supplied version unchanged. The application can expose server log contents through a public route, and the OTP gate may be bypassable depending on middleware registration in files not included in the archive.

The overall production-readiness score is **4.5/10**. The user interface and code organization are ahead of the security and domain-state implementation. Fixing the first six items in the remediation sequence at the end of this report would materially improve readiness.

Area	Rating	Assessment
Architecture	7 / 10	Good direction; several earlier audit issues were refactored.
Authentication / Security	4 / 10	OTP and debug exposure are production blockers.
Authorization	6 / 10	Owner/customer protection is generally good; role model needs cleanup.
Reservation / Pre-order Logic	3 / 10	Pure Reservation and automated 24-hour policy are not fully implemented.
UI / UX	7 / 10	Strong responsive work, but requirements remain inconsistent across pages.
Data Integrity	5 / 10	Transactions exist, but several state transitions are not idempotent.
Performance	6.5 / 10	Acceptable for small deployment; some queries will degrade with growth.
Production Operations	3 / 10	Public debug facilities and incomplete runnable/test artifacts are blockers.

2. Scope and Review Limitations

The review covered the implementation plans and architectural documentation under `/agent_artifacts` and `/docs`, plus application code in `/app`, `/routes`, and `/resources`. The review also reconciled the newer Pure Reservation / 24-hour requirements and Email OTP requirements with the implementation.

The supplied archive is not a fully runnable Laravel repository. It does not include several normal runtime/build artifacts such as `artisan`, `bootstrap/`, `public/`, `tests/`, `composer.lock`, or the Vite configuration. Therefore, this was primarily a static review. PHP syntax validation could be performed, but commands such as `php artisan test`, `route:list`, and direct verification of middleware aliases in `bootstrap/app.php` could not be completed from the supplied archive.

A version mismatch was also identified: the request described the project as Laravel 11, while the supplied `composer.json` declares `"laravel/framework": "^13.0"`. The deployment and documentation should use one consistent framework version.

3. Production-Blocking Findings

3.1 Public debug-log endpoint

CRITICAL: A route in `routes/modules/inventory.php` exposes application log contents publicly and also invokes an OPcache reset. This should be deleted, not renamed or hidden.

```
Route::get('/debug-logs-temp-2026', function () {
    // Reads recent application log entries
    // and exposes them through a public response.
});
```

Laravel logs can contain exception messages, SQL context, internal file paths, third-party service errors, email addresses, request details, and accidentally logged secrets. A public log viewer is therefore an information-disclosure vulnerability.

3.2 Public stack traces

CRITICAL: Public browse/AJAX endpoints return exception messages and full PHP stack traces to clients.

```
return response()->json([
    'error_debug' => $e->getMessage(),
    'trace' => $e->getTraceAsString()
], 500);
```

Production endpoints should call `report($e)` or log the exception server-side and return a generic error payload to the browser.

3.3 OTP middleware may not be enforcing the second factor

HIGH: The project contains `EnsureOtpVerified.php` and `EnsurePasswordChanged.php`, but the supplied route modules generally show only `auth` or `owner` middleware. Because `bootstrap/app.php` is absent, global registration cannot be conclusively verified.

The normal password-login flow authenticates the user first, sets OTP state to unverified, then redirects to the OTP form. If protected application routes do not also enforce the OTP middleware, a user can manually navigate to an authenticated route and bypass the OTP screen. OTP would then be a redirect convention rather than a security gate.

```
// Protected business routes should effectively require:
['auth', 'otp.verified', 'password.changed']

// Exempt only routes that must remain reachable during the challenge,
// such as OTP verify/resend, logout, and forced password-change pages.
```

3.4 Pure Reservation and 24-hour policy are not implemented end to end

HIGH: The current pre-order and checkout code still contains payment/downpayment concepts and does not include an automated 24-hour reservation-expiration workflow.

The newer requirements say there should be no downpayment and no full payment online, the reservation should be paid on pickup, and unattended reservations should automatically cancel after 24 hours. The code and UI still contain the previous 50% downpayment flow in multiple places.

4. Security and Authentication Review

4.1 OTP verification and resend are not rate-limited

HIGH: OTP verification and resend routes do not show throttling. A six-digit code has only one million combinations and must be protected from repeated guesses and resend abuse.

Recommended controls: server-side attempt limits for verification, a resend cooldown, temporary lockout after repeated failures, and security logging for abnormal OTP activity.

4.2 OTP generation should use a cryptographically secure generator

The OTP controller uses `mt_rand(100000, 999999)`. For an authentication code, use `random_int(100000, 999999)`.

4.3 OTPs are stored in plaintext

HIGH: Active OTP values are stored directly in the user record. A database read would expose valid authentication codes.

Store a hash of the OTP and verify it using Laravel Hash utilities. Also cast `otp_expires_at` explicitly to `datetime` in the User model.

4.4 SMTP delivery failures can strand users

Registration and login catch email-send exceptions and continue to the OTP form. If Queen SMTP is unavailable, the user can reach a challenge screen for a code that was never delivered. The flow should surface a controlled delivery-failure state and provide a safe resend/recovery path.

The OTP mail class imports `ShouldQueue` but does not implement it, so OTP mail is currently sent synchronously. Queueing is optional for a small deployment, but the implementation should be deliberate.

4.5 Queen SMTP configuration cannot be verified from the archive

Laravel mail configuration is compatible with a standard SMTP provider, but the real deployment `.env` is not included. The example environment still uses the normal local/log-style defaults. Before production, confirm the actual Queen SMTP host, port, encryption, credentials, sender identity, SPF/DKIM alignment, and failure logging.

4.6 Google login bypasses the Email OTP flow

HIGH: The Google OAuth controller logs the user in and redirects to the dashboard without generating an OTP or explicitly marking OTP verification incomplete.

The security specification should decide whether Google authentication itself is accepted as the second factor. Based on the Phase 5 wording requiring Email OTP on sign-in, Google users should also pass the OTP challenge unless the requirement is revised.

4.7 Password login throttling is substantially correct

The login request uses Laravel RateLimiter checks and a 30-second hit duration, implementing approximately five failed attempts followed by a short lockout. That is a good server-side control. The UI does not appear to show a synchronized countdown/disabled input state, but that is a usability issue rather than the primary security control.

4.8 Unsafe database role default

HIGH: A migration defines the user role default as `owner`. Any future user creation path that forgets to set a role could therefore accidentally create an owner.

```
// Risky default
$table->string('role')->default('owner');

// Safer direction
$table->string('role')->default('customer');
// or remove the database default and require an explicit value.
```

Current normal registration paths appear to explicitly set the customer role, which reduces immediate exploitability, but secure defaults should fail closed. Consider also removing `role` from general mass-assignable attributes.

4.9 Role model does not fully match documentation

The Phase 5 plan references owner, admin, and customer. The supplied code primarily implements owner and customer behavior. If owner and admin are intended to be the same person, the documentation should say so. If they represent different permission scopes, RBAC is incomplete.

4.10 CSP is present but weakened

The project includes a security-headers middleware, which is a positive control, but its CSP permits `unsafe-inline` and `unsafe-eval` for JavaScript. These allowances weaken CSP protection against XSS. Long-term, move inline scripts into the Vite bundle and use nonces/hashes where necessary. As with OTP middleware, global activation cannot be confirmed without the missing bootstrap file.

5. Reservation, Pre-order, and 24-Hour Business Logic

5.1 Pre-order flow still contains old payment logic

HIGH: The pre-order controller still calculates a downpayment and can create an external payment invoice. Admin actions and UI labels also retain "Down Payment Received" behavior.

Historical database columns such as `down_payment`, `is_paid`, `xendit_invoice_id`, and `xendit_invoice_url` can remain for backward compatibility, but new Pure Reservation transactions should not depend on them.

5.2 Client-controlled price in pre-order request

HIGH: The Blade form submits price in a hidden field and the controller reads that request value. Hidden inputs are not trusted server data; users can alter them through browser tools or custom HTTP requests.

```
// Client should submit only identifiers and safe intent data:
inventory_item_id
quantity

// Server should load authoritative values:
$item = InventoryItem::findOrFail($request->inventory_item_id);
$price = $item->price;
$partName = $item->part_name;
$model = $item->model;
```

The same trust boundary applies to product name/model fields. The server should derive product attributes from the database, not from hidden form values.

5.3 Stale downpayment JavaScript can throw a runtime error

The pre-order form queries `document.getElementById('down_payment')` and later writes to the element, even though the corresponding field is no longer present. This can produce a null-reference JavaScript error. Because the new business rule eliminates downpayment, the old calculation block should be removed rather than repaired.

5.4 The 24-hour policy currently exists as UI text, not server behavior

HIGH: The user-facing pages state that reservations automatically cancel after 24 hours, but the supplied database/model/console code does not show a reservation-expiry timestamp or scheduled expiration job/command.

A robust implementation should store server-controlled lifecycle timestamps and expire reservations through a scheduled command or queued job. Expiration must be transactional and must release reserved inventory exactly once.

```
Suggested lifecycle fields / states:

status: pending | processing | ready_for_pickup | picked_up | cancelled | expired
reserved_at
ready_at
expires_at
picked_up_at
```

5.5 Clarify when the 24-hour window starts

For an in-stock reservation, a 24-hour deadline from reservation creation is reasonable. For an out-of-stock pre-order, cancelling 24 hours after the request is created may make no sense if procurement takes several days. A better rule is: in-stock reservation expires 24 hours after reservation; out-of-stock pre-order expires 24 hours after it becomes ready and the customer is notified. This should be explicitly stated in both documentation and code.

5.6 Checkout records reservations as completed sales

HIGH: The order commit flow creates a Sale, while the sales status migration defaults to `completed`. A pay-at-pickup reservation can therefore be recorded as a completed sale before payment occurs.

The checkout flow also records the grand total as amount tendered when no cash amount is supplied. This can overstate revenue and create misleading receipts/reporting.

A reservation/order should have its own lifecycle and should become a completed Sale only when the customer actually pays and picks up the item.

5.7 Customer cancellation conflicts with the sale default

Customer cancellation allows only a `pending` sale, but newly created online checkout records default to `completed`. Therefore the cancellation workflow can be unreachable for freshly created reservations. This is another symptom of mixing reservation state and sale state in the same entity.

5.8 Pre-order completion is not idempotent

HIGH: The administrative completion path increments inventory and then updates the pre-order. Without a strict state-transition guard and a row lock on the pre-order, duplicate submissions or concurrent requests can increment stock multiple times.

Every stock-changing transition should validate the current state inside a database transaction, lock the relevant records, and reject repeated execution after the transition has occurred.

5.9 Stock reservation semantics should be clearer

Checkout currently decreases the physical quantity immediately. That can work as a simple reservation model only if every cancellation and expiry reliably restores stock. A cleaner model is to track on-hand and reserved quantities separately:

```
quantity_on_hand
quantity_reserved
quantity_available = quantity_on_hand - quantity_reserved

Reservation created: reserved += qty
Reservation cancelled/expired: reserved -= qty
Pickup/payment: on_hand -= qty; reserved -= qty
```

This approach improves auditability and reduces ambiguity between physical stock, sellable stock, and reserved stock.

5.10 User-facing requirements are contradictory

HIGH: The newer UI contains Pure Reservation / 24-hour language, while other pages still instruct customers to make a 50% downpayment through GCash and still display old payment statuses.

The welcome page, FAQ, pre-order terms, custom pre-order screen, and older project documentation should be revised so the customer sees one consistent business rule.

6. Architecture and Laravel Best Practices

6.1 Positive architectural improvements

Several architectural changes directly address weaknesses identified in earlier audits. The following are strong improvements:

- `CartRepository` abstracts session-cart behavior.
- `PaymentGateway` provides a payment abstraction rather than hard-coding a provider everywhere.
- `CommitOrder` centralizes transaction/order commit behavior.
- `InventorySearchService` extracts search logic from controllers.
- Policies exist for Inventory, PreOrder, Repair, and Sale resources.
- FormRequests are used for validation in important flows.
- Database transactions and `lockForUpdate()` appear in stock-sensitive paths.
- ULID transaction references reduce predictable identifier risk.
- Routes are split into modules, improving maintainability.
- Receipt/customer ownership checks reduce IDOR risk.

The architecture is therefore not the primary reason for the low production score. The largest remaining issue is domain modeling: reservations, pre-orders, payments, pickups, and completed sales are still partially mixed together.

6.2 Modular routes are a good direction

The route files are divided by functional area and generally apply owner/customer middleware consistently. This is easier to maintain than a single large route file. The main concern is ensuring that security middleware such as OTP verification is attached to the correct groups globally or explicitly.

6.3 Multi-shop scaffolding is not yet scoped

The code contains `shop_id`-style scaffolding that is not consistently query-scoped. This is not an immediate blocker for a permanently single-shop deployment. If MotoTrack becomes multi-tenant, every query, policy, relationship, and unique constraint that touches tenant-owned data must be scoped to the active shop.

7. UI/UX and Frontend Review

7.1 Navigation and responsive layout are substantially improved

The navigation is sticky, the main desktop links are centered, account/theme/notification controls are separated to the right, and the mobile navigation opens from the right side. This aligns well with the newer design requirements.

The customer dashboard also uses a responsive multi-column layout that places the welcome content and Recent Updates side-by-side on larger screens. The admin Live Activity feed is limited to the overview area, which matches the stated requirement.

Tables generally use horizontal overflow wrappers, which is appropriate for smaller screens.

7.2 "Single-screen" overview is not guaranteed

The dashboard still relies on normal document flow, `min-h-screen`, padding, and content height. It does not calculate a fixed desktop content viewport or consistently constrain internal panels. Therefore, fitting the complete overview without page scrolling is not guaranteed on common 1366x768 laptops, smaller heights, or non-default browser zoom.

Do not globally force `overflow-hidden`, because that can hide content. Instead, use bounded desktop panel heights and internal scrolling only for feeds/tables where appropriate.

7.3 Alpine Collapse plugin appears missing

MEDIUM: The Blade navigation uses the `x-collapse` directive, but the supplied JavaScript imports Alpine itself without clearly registering the Alpine Collapse plugin.

```
import collapse from '@alpinejs/collapse';
Alpine.plugin(collapse);
```

Either install/register the plugin or remove the directive. Otherwise mobile submenu animation/behavior can be inconsistent.

7.4 Tailwind major versions appear mixed

The package configuration includes Tailwind 3 and the Tailwind/Vite integration for Tailwind 4, while the CSS uses Tailwind 3-style directives. Standardize on one Tailwind major version and ensure the Vite integration matches it. The supplied archive does not include the Vite configuration, so the active build path could not be verified.

8. Authorization and Endpoint Review

8.1 Owner and customer access controls are generally well-scoped

Owner administration routes are consistently grouped under owner middleware, and resource policies exist for key models. Customer receipt lookup also scopes data to the authenticated customer, which is an important protection against insecure direct object references. Customer order cancellation/refund code similarly checks ownership.

8.2 Formal REST API is not present in the supplied archive

No `routes/api.php` was included. The Professional Features Guide appears to describe a future or conceptual API architecture rather than an implemented public REST API. Existing JSON endpoints are primarily browser/AJAX endpoints.

For those endpoints, return only fields required by the frontend, apply sensible limits, remove stack traces, and introduce API Resources if they evolve into a formal external API.

8.3 Search throttling is present but generous

Public search/suggestion endpoints include throttling, which is positive. The current request allowance is relatively high for anonymous search. This is acceptable initially but should be observed under real traffic and tightened if scraping or abuse becomes visible.

9. Performance and Scalability Observations

For a small single-shop deployment, the application should be workable from a performance perspective. The main opportunities are query efficiency and keeping dashboard payloads bounded as data grows.

9.1 Notification queries in Blade

The navigation accesses notification relationships directly in Blade. Calling `auth()->user()->notifications->take(10)` loads the relationship collection before taking ten items. Prefer database-level `count()`, `latest()`, and `limit(10)` queries, ideally prepared outside the view.

9.2 Dashboard history can grow without bounds

The customer dashboard retrieves complete repair/pre-order/purchase collections in several places. This will be fine for a new system with tens of records but will become increasingly expensive. Use recent-record limits on the overview and pagination/lazy loading on history screens.

9.3 Existing positive performance controls

Database aggregation is used for several dashboard totals, and dashboard statistics are cached for a short interval. These are good choices. Row locking in stock-sensitive transactions also favors correctness under concurrency.

10. Recommended Production Remediation Sequence

1. **Delete the public debug-log route** and remove stack traces/exception details from all public responses.
2. **Make OTP a mandatory middleware gate** for business routes; include Google login in the chosen second-factor policy; throttle OTP verification/resend; hash OTPs; use `random_int()`.
3. **Replace the remaining payment/downpayment pre-order flow with a true reservation state machine.**
4. **Implement server-side 24-hour expiration** with scheduled execution, transactional stock release, and clear rules for in-stock reservations vs. out-of-stock pre-orders.
5. **Separate reservation/order state from completed Sale state** so revenue and receipts represent real completed payments rather than pending pickup reservations.
6. **Make every stock/status transition transactional and idempotent**, especially pre-order completion, cancellation, expiry, and restocking.
7. **Remove all remaining 50% downpayment wording and obsolete payment statuses** from Blade files, FAQ, terms, custom pre-order UI, and final documentation.
8. **Change the unsafe default user role** and finalize whether "admin" is distinct from "owner".
9. **Normalize Alpine and Tailwind dependencies**, then test all relevant mobile/tablet/desktop breakpoints.
10. **Restore/include the complete Laravel runtime repository and automated tests**, then test OTP bypass, authorization, concurrent reservation, expiry, cancellation, stock restoration, and duplicate submissions.

11. Minimum Test Coverage Before Production

The following feature/integration tests are particularly important because they directly cover the high-risk areas identified in the review:

- Authenticated but OTP-unverified user cannot access dashboard, cart, checkout, pre-order, order-history, or owner routes.
- OTP verify and resend endpoints throttle correctly and expired/used OTPs cannot be replayed.
- Google sign-in follows the documented second-factor policy.
- A customer cannot submit a manipulated product price/model/name and affect server-side totals.
- Two concurrent users cannot reserve more inventory than is available.
- Double-clicking a status/completion action cannot restock or charge/convert the same transaction twice.
- Reservation expiry runs once, releases reserved stock once, and records the correct terminal state.
- Customer cancellation releases reserved stock once and cannot cancel another customer's record.
- Reservations do not count as completed sales/revenue until pickup/payment is finalized.
- Public endpoints never expose stack traces, debug logs, credentials, internal paths, or exception details.

12. Final Verdict

MotoTrack is a credible project with a noticeably improved architecture and a UI that already satisfies several of the newer design requirements. The strongest parts are modularization, transactional stock handling in several paths, policy-based access control, custom dashboard work, and service/repository abstractions.

The current version is nevertheless **not production-ready**. The release should be blocked until the public debug exposure is removed, OTP is proven to be mandatory rather than redirect-only, the Pure Reservation state model is implemented, and the 24-hour policy is enforced server-side. These are not cosmetic improvements; they directly affect confidentiality, authentication assurance, inventory integrity, and financial reporting.

After the first six remediation items are completed and covered by automated tests, the platform could reasonably move toward approximately 7.5-8/10 production readiness, subject to a final runtime/deployment review of the complete Laravel repository and production configuration.

Appendix A. Key Files Referenced

- `composer.json`
- `routes/modules/inventory.php`
- `routes/modules/customer.php`
- `routes/modules/shop.php`
- `routes/modules/preorders.php`
- `routes/auth.php`
- `routes/console.php`
- `app/Http/Middleware/EnsureOtpVerified.php`
- `app/Http/Middleware/EnsurePasswordChanged.php`
- `app/Http/Controllers/Auth/OtpController.php`
- `app/Http/Controllers/Auth/GoogleController.php`
- `app/Http/Controllers/PublicBrowseController.php`
- `app/Http/Controllers/CustomerPreOrderController.php`
- `app/Http/Controllers/AdminPreOrderController.php`
- `resources/views/layouts/navigation.blade.php`
- `resources/views/dashboard.blade.php`
- `resources/views/welcome.blade.php`
- `resources/views/faq.blade.php`
- `resources/views/preorders/form.blade.php`
- `resources/views/shop/checkout.blade.php`
- `resources/js/app.js`
- `database/migrations/2026_04_13_094212_add_role_to_users_table.php`
- `database/migrations/2026_09_22_030154_add_status_to_sales_table.php`
- `agent_artifacts/implementation_plan_v2.md`
- `agent_artifacts/phase_5_security_plan.md`
- `docs/* architectural audits and system requirements PDFs`

Note: This report reflects the supplied archive and the static evidence available in it. Items depending on omitted runtime files, environment configuration, queue/scheduler configuration, or deployment infrastructure should be re-verified against the complete production repository before launch.